轻量化图模式后端 npugraph_ex

作者：徐道鸿

时间：2026.02.10

npugraph_ex：大模型推理“低时延”需求解决方案

背景

随着人工智能的飞速发展，大模型推理场景对“低时延”提出了苛刻要求。PyTorch eager mode虽然灵活，但下发算子任务的调度开销成为性能瓶颈。

业界方案

随着PyTorch2.0的发布，其核心功能之一 torch.compile，通过即时编译将Python代码转换为计算图，为解决时延问题提供了关键工具

npugraph_ex

基于torch.compile，提供一个NPU亲和的图模式后端，并集合ACLGraph图下沉能力，旨在提升易用性、提供较好的NPU开箱性能。

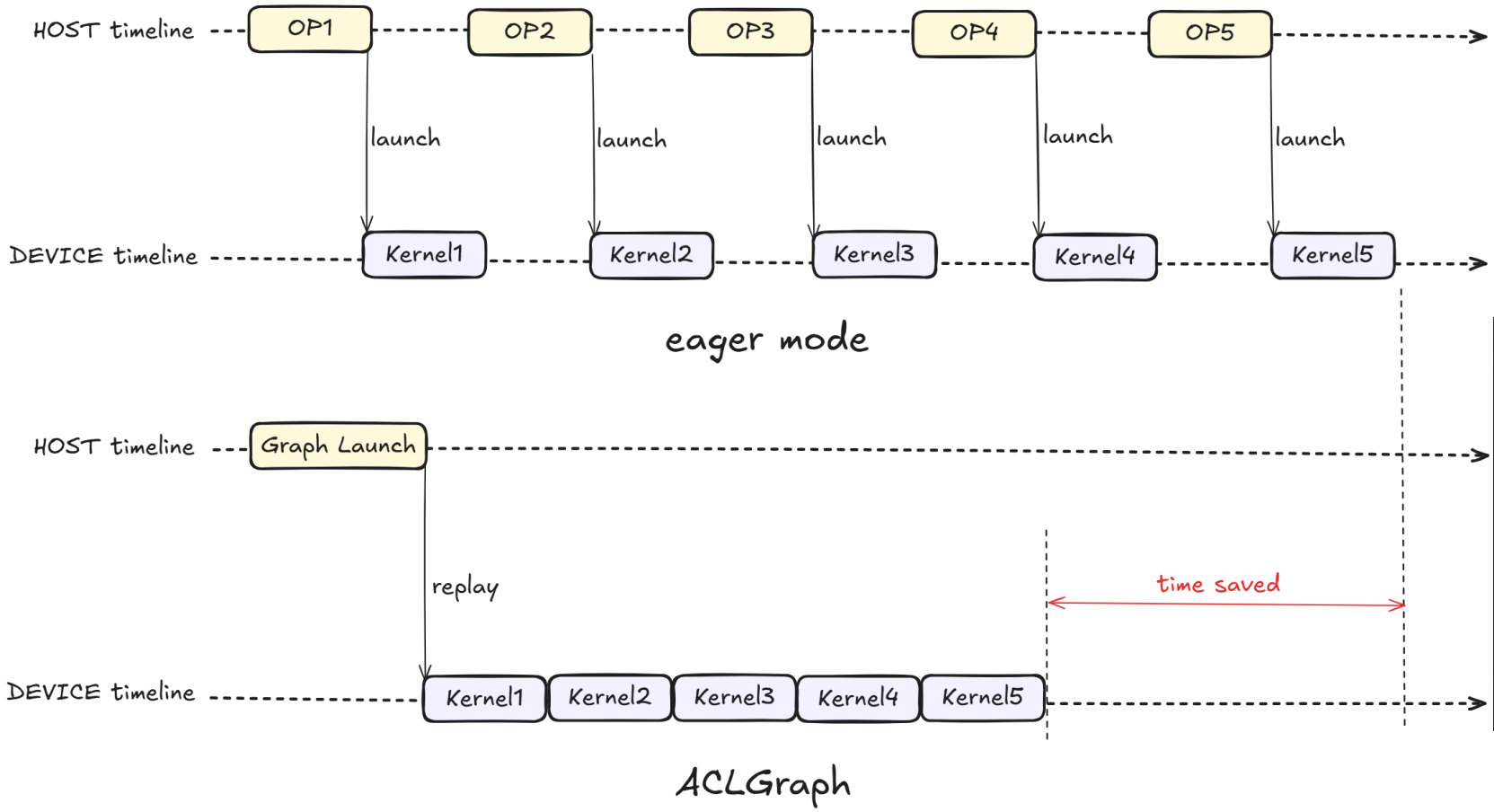
快速接入：CANN提供ACLGraph，有效的消除了算子调度时延，并已经集成在PyTorch中，可快速使能；

高性能：不止于图下沉，在复用ACLGraph调度能力的基础上，进一步在PyTorch FX图层面叠加NPU亲和的深度优化。

可集成：面向主流服务化框架设计，可无缝、快速地对接到vLLM、SGLang等生态中。

ACLGraph：整图下沉优化调度时延

ACLGraph是CANN提供，通过1次capture+多次replay的方式减少算子下发调度开销，提升host bound场景端到端执行性能。

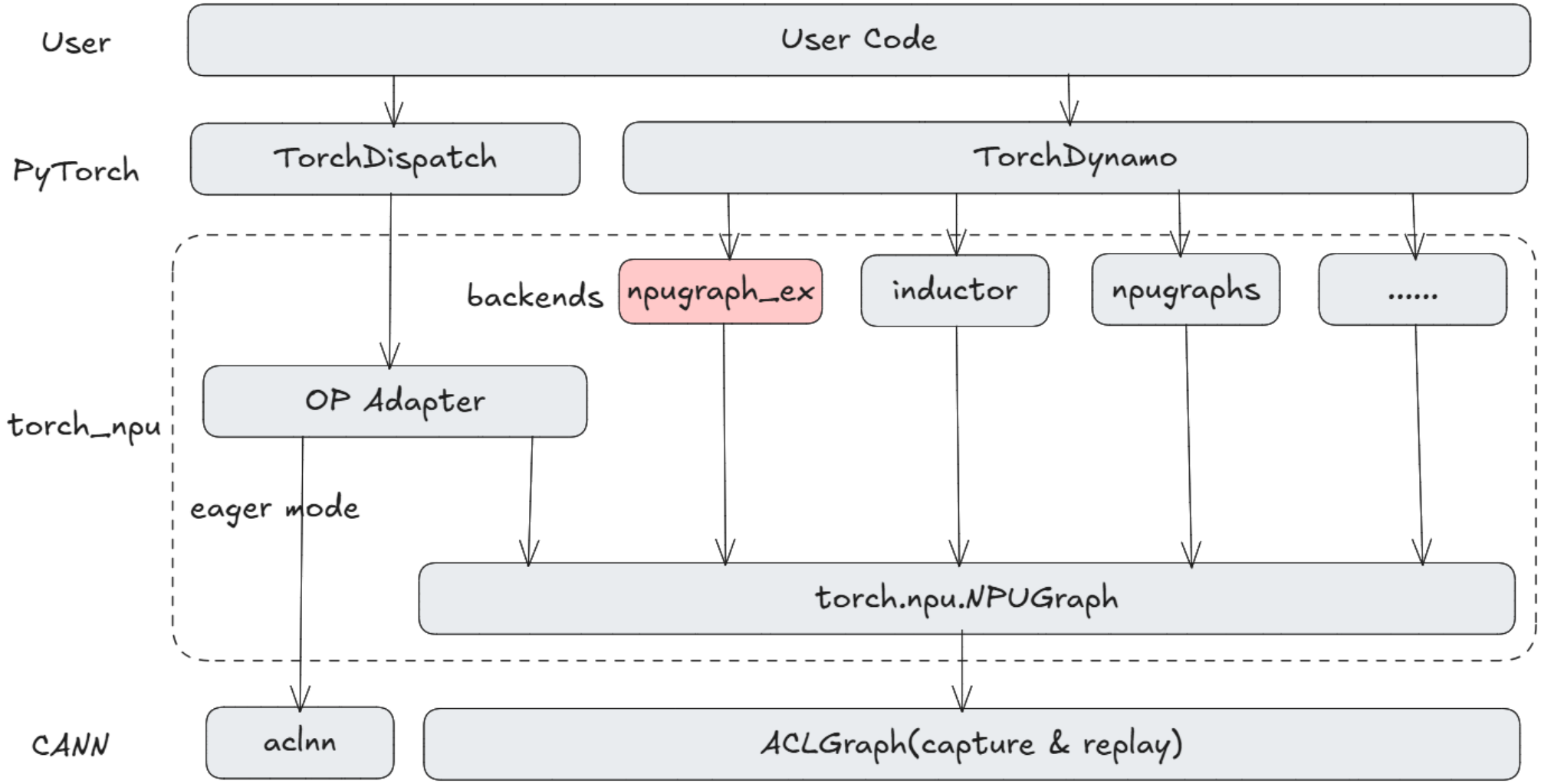



```python
import torch
import torch_npu

static_input = torch.randn(128, 128, device="npu")

g = torch.npu.NPUGraph()
with torch.npu.graph(g):
  # capture
  output = model(static_input)

# replay
g.replay()
```

https://gitcode.com/cann

CANN

整体架构：npugraph_ex在PyTorch编译流程中的位置



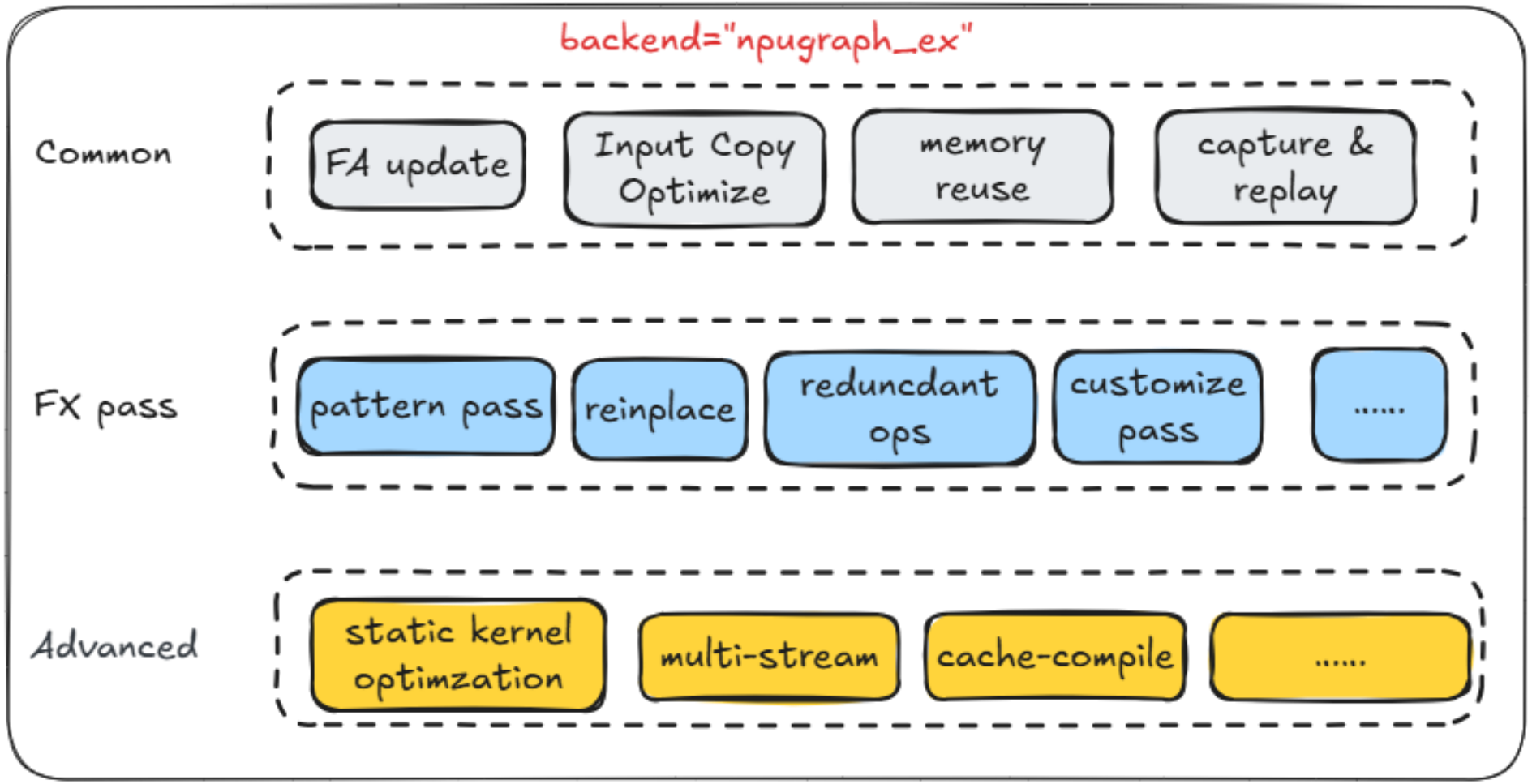
使用方式：

```
torch.compile(model, backend="npugraph_ex")
```

https://gitcode.com/cann

CANN

npugraph_ex 核心技术总览



基础能力：
使用ACLGraph提升**调度性能**的同时，优化**内存复用**。

图优化：
基于FX graph做算子融合、冗余节点消除等**图级别**优化。

深度优化：
在开箱性能的基础上，通过特定适配，获取更多性能优化。

CANN

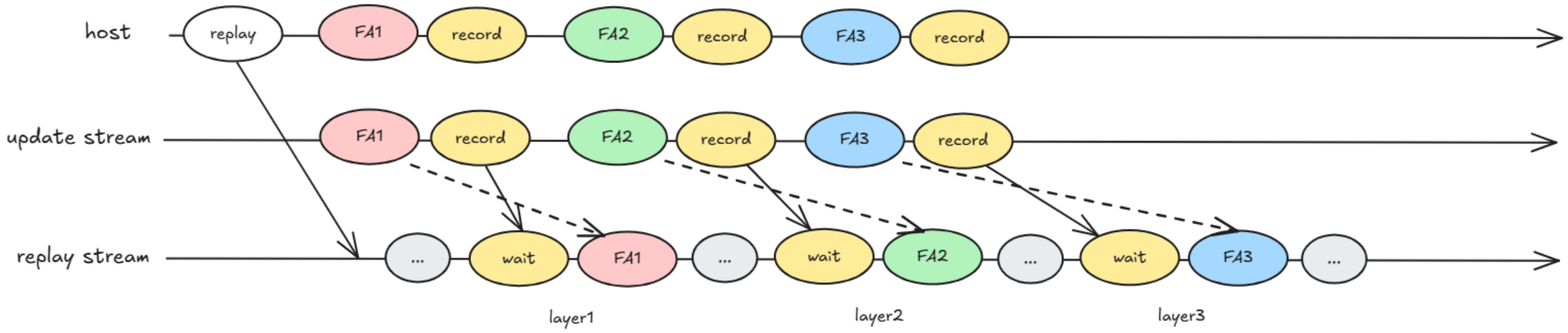
核心特性1：host tiling 参数刷新实现FA下沉

背景：FlashAttention类的算子需要在host上计算tiling，tiling计算与句长相关且动态变化，单纯的replay无法实现参数的动态刷新，导致整网精度问题。

ACLGraph提供在Replay阶段刷新FA算子tiling信息的能力：

- **Capture阶段：**记录需要参数更新的算子，并在ACLGraph前插入Event wait任务控制时序；
- **Replay阶段：**ACLGraph replay后，下发参数更新任务，并在每个更新任务后插入Event record，确保更新顺序。

npugraph_ex将上面两个阶段封装为FX pass，自动识别图中的FA算子，用户无需感知，即可获取整图下沉带来的性能受益。



CANN

核心特性2：精细内存管理提高内存复用降低用户使用门槛

背景：图下沉带来调度性能受益的同时，因为replay阶段需要固定内存地址，带来了内存管理的复杂性。

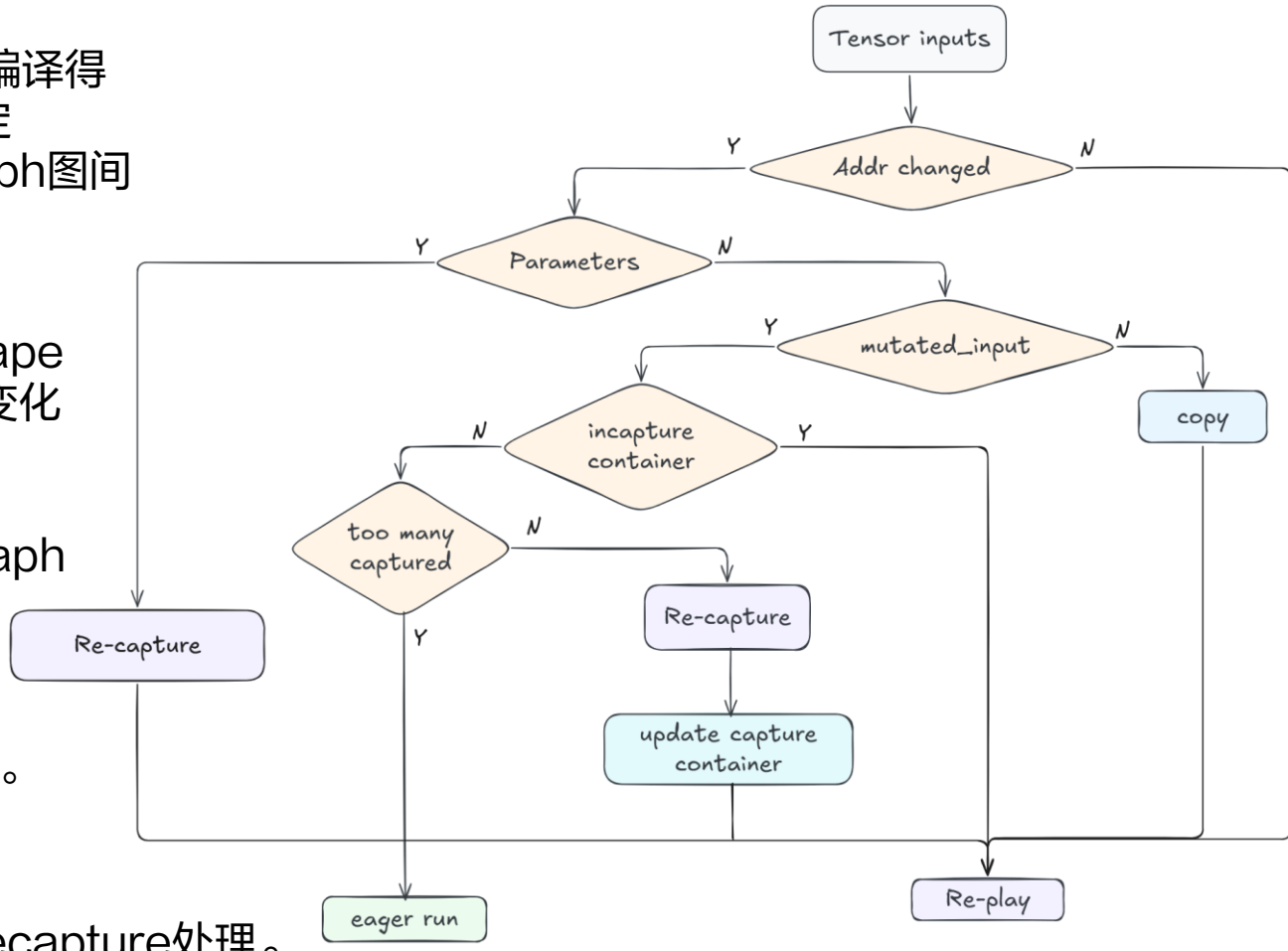
图间内存复用：

dynamic= False

一个用户模型，不同输入shape编译得到多张FX Graph，允许用户指定graph_pool，达到多张FX Graph图间内存复用的目的。

dynamic= True

一个用户模型，编译得到动态shape的FX Graph，因为输入shape变化会capture多张ACLGraph图，npugraph_ex内部会使用同一graph_pool，达到多张ACLGraph图间内存复用的目的。



输入拷贝优化

识别模型输入类型，保证正确性的前提下，减少拷贝与内存开销。

模型输入： 内存地址发生变化，拷贝。

Parameter & mutated_input：内存地址发生变化，自动做recapture处理。

通过以上精细化管理，保证正确性的同时，减少内存开销。

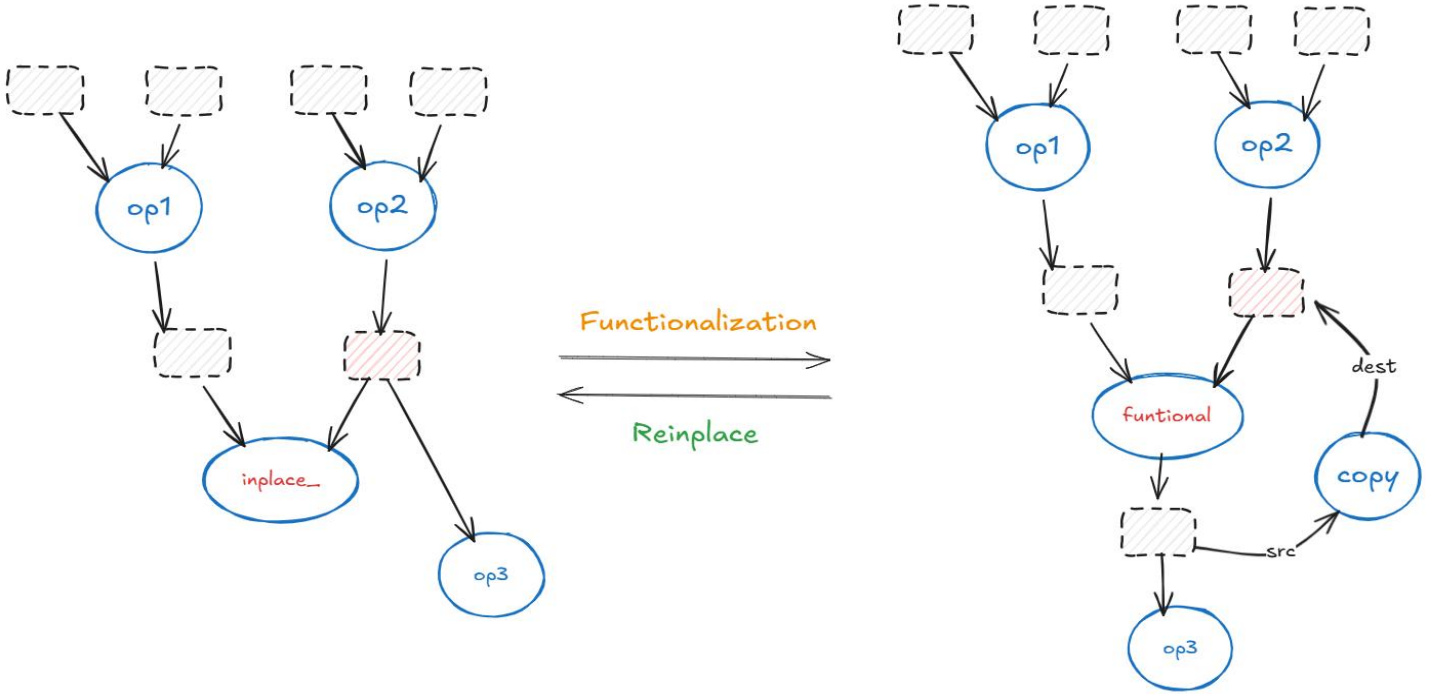
CANN

核心特性3：reinplace pass，减少内存提升性能

背景：函数化在带来图优化便利的同时也带来性能和内存的负面影响

Funtionalization：AOTAutograd 做 Functionalization 是为了消除 in-place 和 alias 带来的副作用，把 forward 转换成纯函数形式，使 forward/backward 能被提前、正确地生成和编译，方便AOT 自动求导和后续编译器做优化。

Reinplace：与Funtionalization相反，编译器做完其他优化后将in-place算子替换回去，消除多余的内存搬运，提升性能。




```
Python forward (user code)
│
│  include in-place / view / alias
▼
TorchDynamo
│  FX Graph
▼
AOTAutograd
┌──────────────────────────────┐
│     Functionalization        │  ⭐ 关键步骤
│  - remove in-place           │
│  - alias / view              │
│  - forward -> functional     │
└──────────────────────────────┘
▼
Inductor / Other Backend
│  - fusion
│  - memory planning
│  - codegen
▼
Compiled forward + backward
```

https://gitcode.com/cann

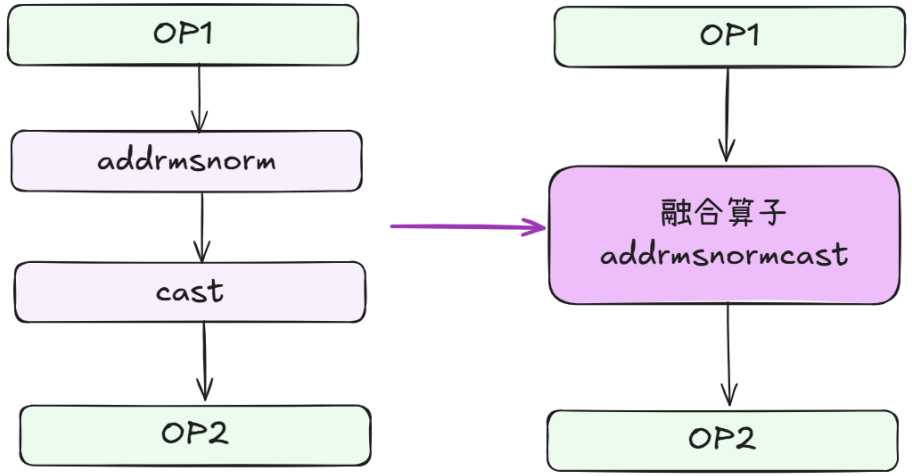
npugraph_ex通过reinplace消除函数化的副作用，为下一步图优化扫出障碍。

CANN

核心特性4：FX pass基于整图优化，提升性能

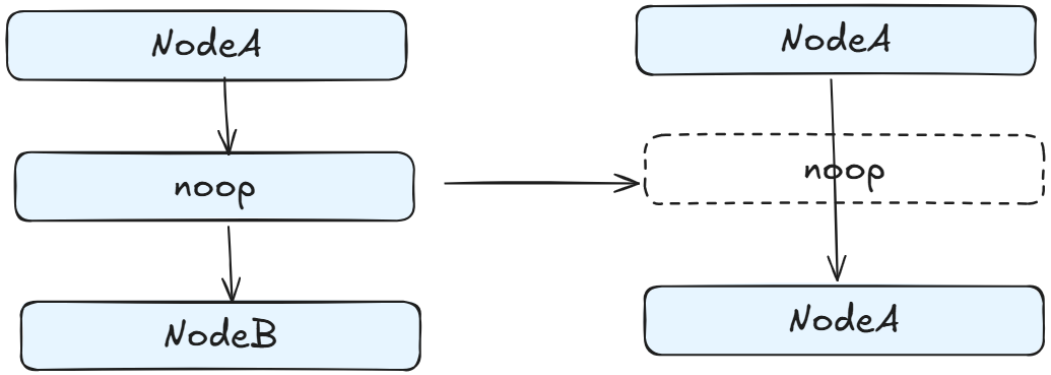
- **pattern pass**

将多个离散算子融合成一个高效的融合算子。



- **冗余算子消除**

自动识别并消除图中不影响计算逻辑的冗余操作。



- **自定义pass**

开放API，允许开发者在FX Graph中直接自定义pass：

register_replacement(search_fn, replace_fn, example_inputs, trace_fn=fwd_only, extra_check=_return_true, search_fn_pattern=None)

search_fn：该函数是希望在 FX 图中识别的算子组合或计算逻辑，如需要融合的算子组合。

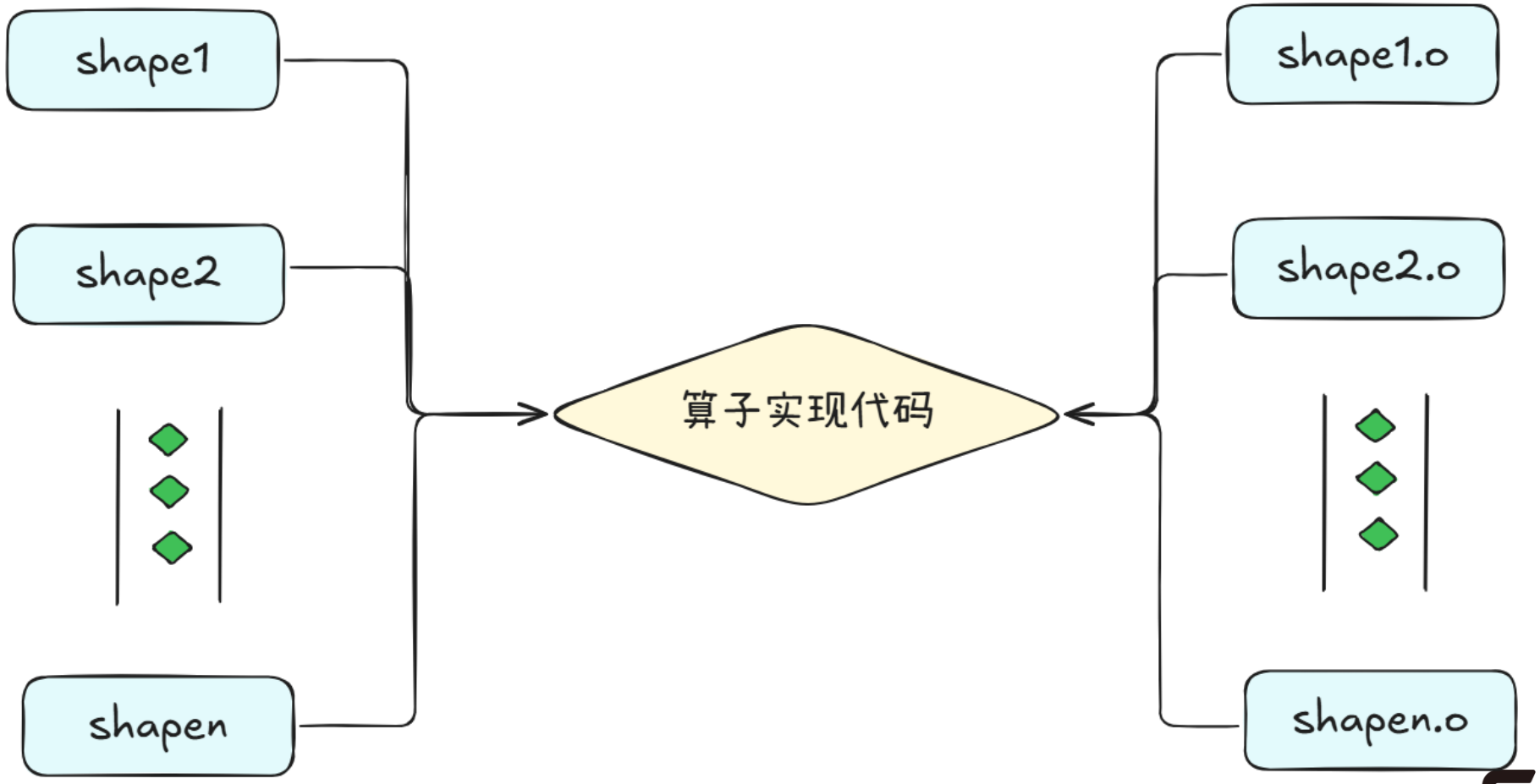
replace_fn：在目标图中找到 search_fn 对应的组合时，会用这个函数的计算逻辑替换原有子图，实现算子融合或优化。

example_inputs：用于追踪 search_fn 和 replace_fn 的示例输入张量。输入的形状和 dtype 需与实际场景匹配。

CANN

进阶特性1：静态kernel编译 提升算子执行性能

对于纯静态shape网络或者shape变化较少的动态shape网络，可以在capture前根据输入shape，提前编译生成静态kernel。通过编译器在静态shape下的优化，获得执行性能更好的算子kernel。



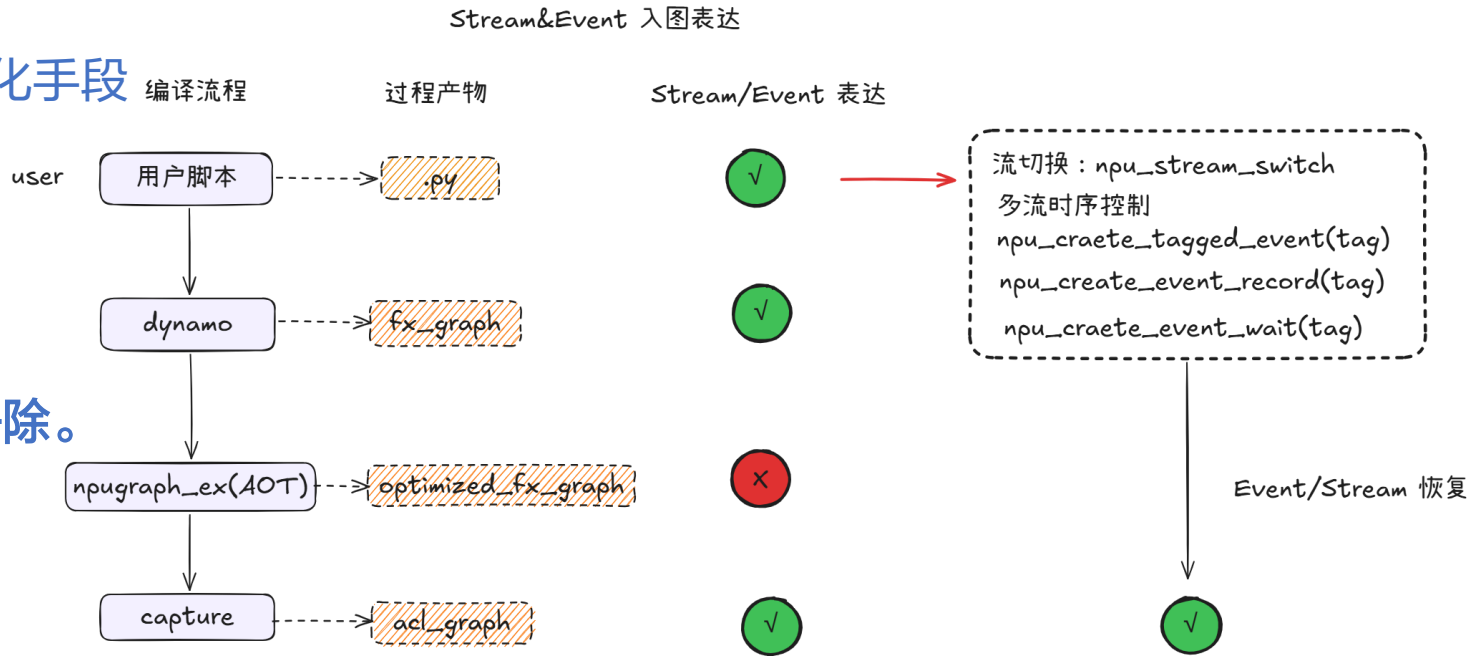
CANN

进阶特性2：多stream支持 最大化硬件利用率

场景：推理场景，计算通信并行是常用的性能优化手段

- **多stream支持**

npugraph_ex基于AOTAutograd工作，Stream/Event表达均会被functionalize功能去除。



- **算子限核能力**

多流场景下，会出现所有核都被一个流占用的情况，导致算子执行并行度降低，因此需要把核分给不同的流用，从而保证算子并行执行收益。

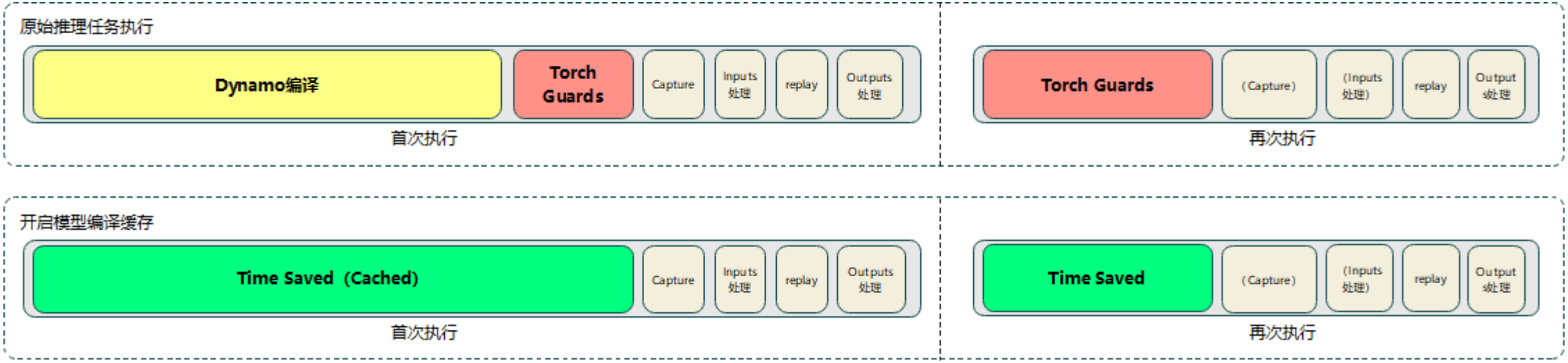
```
class Model(torch.nn.Module):
    def __init__(self):
        super().__init__()

    def forward(self, in1, in2, in3, in4):
        add_result = torch.add(in1, in2)
        with torch.npu.npugraph_ex.scope.limit_core_num(2, 4):
            mm_result1 = torch.mm(in3, in4)
            with torch.npu.npugraph_ex.scope.limit_core_num(12, 24):
                mm_result2 = torch.mm(in3, in4)
        return add_result, mm_result1, mm_result2
```

多stream配合算子限核，通过并发提高硬件利用率，获得更好的性能。

CANN

核心特性3：编译缓存优化冷启动开销

背景：生产环境部署对服务的冷启动时间有较高要求，图编译耗时成为了影响部署的关键瓶颈。




```python
def cache_compile(
    func, # 缓存编译的method，只支持module的method
    *,
    config: Optional[CompilerConfig] = None, # 图编译配置
    dynamic: bool = True, # 是否按照输入动态trace
    cache_dir: Optional[str] = None, # 缓存根目录，默认.torchair_cache，权限0o600
    global_rank: Optional[int] = None,
    # 分布式训练时的rank，默认torch.distributed.get_rank()
    tp_rank: Optional[int] = None, # 指定的tp rank
    pp_rank: Optional[int] = None, # 指定的pp rank
    custom_decompositions: Optional[dict] = None, # 用户自定义的decompose策略
    ge_cache: bool = False, # 是否开启GE缓存
    **kwargs) -> Callable:
```

https://gitcode.com/cann

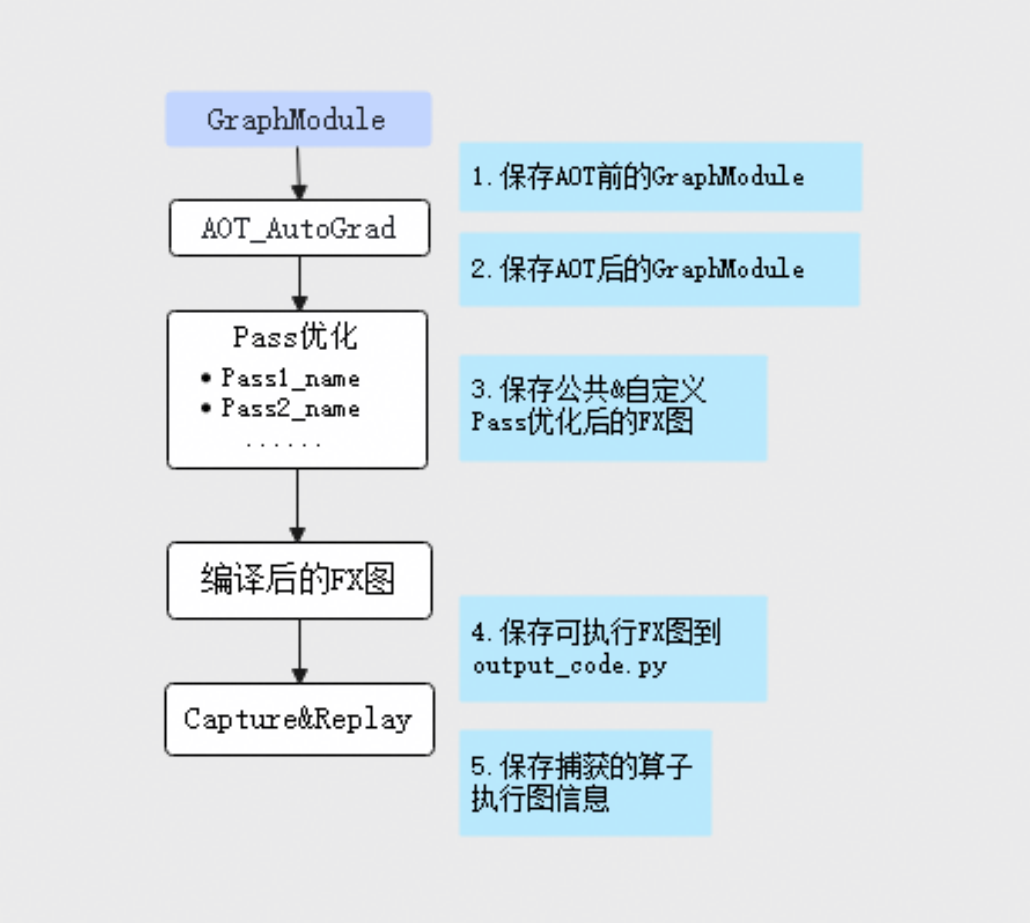
CANN

维测能力1：图编译过程信息保存方便精准定位问题阶段

复用PyTorch原生环境变量 export TORCH_COMPILE_DEBUG=1，与社区Debug信息保存格式一致，方便刚接触npugraph_ex的开发人员高效获取了解图优化过程及问题定位。

```
torch_compile_debug/run_<时间>-pid_<进程号>
├── torchair
│   ├── debug.log                                          # Python和C++层日志
│   ├── model__0                                           # model__0为模型ID
│   │   ├── forward                                        # 前向推理
│   │   │   ├── output_code.py                             # 编译后的图结构文件
│   │   │   ├── 000_aot_forward_graph.txt                  # AOT后的GraphModule
│   │   │   ├── 001_aot_forward_graph_after_${pass1_name}.txt   # 公共图优化过程中每个Pass的输出FX图
│   │   │   ├── 002_aot_forward_graph_after_${pass2_name}.txt
│   │   │   ├── 003_aot_forward_graph_after_${pass5_name}.txt   # aclgraph优化中不同pass处理后的FX图
│   │   │   ├── 004_aot_forward_graph_after_${pass6_name}.txt
│   │   │   ├── ......                                     # 其他Pass优化
│   │   ├── dynamo_out_graph.txt                           # AOT前的GraphModule
│   │   ├── graph_1_id_${aclgraph_id}_rank_${rank_id}_pid_${pid}_ts_${timestamp}.json      # 捕获的算子执行图信息
└── torchdynamo
    └── debug.log                                          # Torch原生Dynamo日志
```

日志信息	1. PyTorch原生Dynamo日志 2. TorchAir Python层日志
Debug信息	1. AOT前的GraphModule 2. AOT后的GraphModule 3. 公共Pass图优化过程中每个Pass的输出FX图（txt文件） 5. aclgraph优化中不同Pass处理后的FX图（txt文件） 6. aclgraph在Capture阶段捕获的算子执行图信息（*.json文件） 7. aclgraph编译后的FX图结构信息（output_code.py文件）



https://gitcode.com/cann

CANN

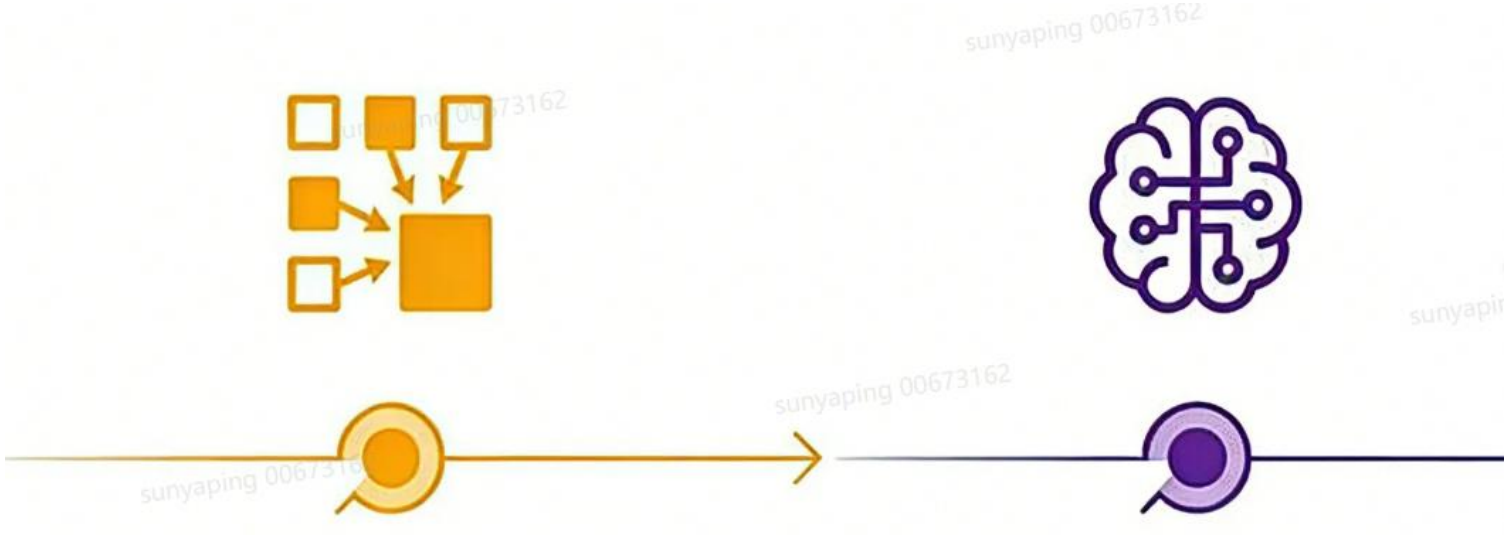
维测能力2：图间数据保存方便整网精度问题定位

背景：在图模式下，由于Python原生print函数会触发同步，导致dynamo断图，因此图模式下无法使用print观察图执行过程中的tensor值。

为了在ACLGraph图下沉场景快速定位精度问题，torch_npu提供了一个类似原生print特性且不影响graph replay的API，允许将ACLGraph中间节点的tensor数据、数据类型、shape信息保存到指定的bin文件中，以便获取图中间计算结果，快速定位精度问题。

```python
import torch
import torch_npu

class Model(torch.nn.Module):
    def __init__(self):
        super().__init__()

    def forward(self, x):
        x = torch.add(x, 2)
        torch_npu.save_npugraph_tensor(x, "/tmp/dump/x_1.bin")
        x = torch.add(x, 2)
        torch_npu.save_npugraph_tensor(x)
        return x

model = Model()
model = torch.compile(model, backend="npugraph_ex", dynamic=False, fullgraph=True)
x = torch.randn([10, 10]).npu()
output = model(x)
```

未来路标：面对低时延，性能持续演进



Super Kernel

集成Super Kernel技术，将图中多个kernel在设备端融合成 一个更大的kernel，极大减少内核切换，提升执行效率。

Graph-Autofusion

实现更加智能的自动算子融合，结合硬件特性自动探索最优的图切分与融合策略，进一步压榨硬件性能。

CANN

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.

查看项目

欢迎star、issue、pr贡献~

CANN TorchAir小助手

扫一扫上面的二维码图案，加我为朋友。

添加TorchAir小助手加入torchair技术交流群

https://gitcode.com/cann

CANN